

Servidor web

Tema 7

Prof. Ing. Angel Brito Segura

1. Introducción

Un servidor web es un software y hardware que utiliza el Protocolo de Transferencia de Hipertexto (HTTP, por sus siglas en inglés) y otros protocolos para responder a las peticiones de los clientes realizadas a través de la World Wide Web (Internet) o en una intranet. Su principal función es distribuir contenidos (documentos) a los usuarios que los solicitan mediante navegadores web u otros clientes HTTP.

En cuanto al hardware de un servidor web se refiere al servidor que almacena el software y los archivos que componen un sitio web (documentos HTML, imágenes, hojas de estilo CSS y archivos JavaScript). Al conectarse a una red, permite el intercambio físico de datos con otros dispositivos conectados.

El software de un servidor web incluye varios componentes que controlan cómo los usuarios acceden a los archivos alojados, principalmente se encuentra un servidor HTTP que es quien interpreta las URL (direcciones web) y este protocolo. Se puede acceder a él a través de los nombres de dominio de los sitios web que aloja y este entrega el contenido de dichos sitios web al dispositivo del usuario.

En su nivel más básico, cuando un navegador necesita un archivo alojado en un servidor web, lo solicita mediante HTTP. Cuando la solicitud llega al servidor web (hardware) correcto, el servidor HTTP (software) la acepta, encuentra el documento solicitado y lo envía de vuelta al navegador mediante el mismo protocolo (si el servidor no lo encuentra, devuelve una respuesta 404).

El desarrollo de los servidores web está estrechamente vinculado al físico e informático británico **Tim Berners-Lee**, quien en 1989 sugirió que el intercambio de información en el CERN (Organización Europea para la Investigación Nuclear) debería realizarse a través de un sistema de hipertexto más fácil y rápido. En 1990, junto con Robert Cailliau, presentó su proyecto **World Wide Web** (inicialmente llamado *Mesh*) construido sobre protocolos TCP e IP existentes, constaba de cuatro componentes básicos:

1. Un formato de texto para representar documentos de hipertexto (Lenguaje de Marcado de Hipertexto, HTML).
2. Un protocolo para intercambiar estos documentos (Protocolo de Transferencia de Hipertexto, HTTP).
3. Un cliente para visualizar (y editar) estos documentos: el primer navegador web, llamado WorldWideWeb.
4. Un servidor para dar acceso a los documentos: **CERN httpd**.

Los primeros servidores funcionaban fuera del CERN a principios de 1991 y el 6 de agosto de este año, Tim Berners-Lee publicó en el grupo de noticias `alt.hypertext`, dando inicio oficial a la World Wide Web como proyecto público.



Figura 1: Tim Berners-Lee (1955 -)

2. Protocolo HTTP

Protocolo para obtener recursos de un servidor como documentos HTML. Es la base de cualquier intercambio de datos en la web y es un protocolo cliente-servidor, lo que significa que las solicitudes las inicia el destinatario, generalmente el navegador web. Un documento completo se compone típicamente de múltiples recursos como texto, instrucciones de diseño, imágenes, vídeos, scripts y más, provenientes de diferentes servidores.

Los clientes y los servidores se comunican intercambiando mensajes individuales (en lugar de un flujo de datos). Los mensajes enviados por el cliente se denominan solicitudes y los mensajes enviados por el servidor como respuesta se denominan respuestas.

Diseñado a principios de la década de 1990, HTTP es un protocolo extensible que ha evolucionado con el tiempo. Su primera versión era muy simple y se le denominó HTTP/0.9: protocolo de una sola línea.

2.1. HTTP/0.9

La versión inicial de HTTP carecía de numeración de versión; posteriormente se denominó 0.9 para diferenciarla de versiones posteriores. Era extremadamente simple: las solicitudes consistían en una única línea que comenzaba con el único método posible, GET, seguido de la ruta al recurso. No se incluía la URL completa, ya que el protocolo, el servidor y el puerto no eran necesarios una vez establecida la conexión con el servidor. La respuesta era igualmente simple: consistía únicamente en el archivo en sí mismo.

A diferencia de evoluciones subsecuentes, no existían cabeceras. Esto implicaba que únicamente se podían transmitir archivos HTML. No existían códigos de estado o de error. Si ocurría un problema, se generaba un archivo HTML específico que incluía una descripción del problema para consumo humano.

2.2. HTTP/1.0

Debido a que la versión inicial de HTTP era muy limitada, se tuvo que agregar más información para que los navegadores y servidores fueran más versátiles:

- Se enviaba información de versionado en cada solicitud (se anexaba HTTP/1.0 a la línea GET).
- Se enviaba también una línea de código de estado al inicio de una respuesta. Esto permitía al propio navegador reconocer el éxito o fracaso de una solicitud y adaptar su comportamiento en consecuencia; por ejemplo, actualizando o utilizando su caché local de una manera específica.
- Se introdujo el concepto de cabeceras HTTP tanto para solicitudes como para respuestas. Se podían transmitir metadatos y el protocolo se volvió extremadamente flexible y extensible.
- Se podían transmitir documentos distintos a archivos HTML planos gracias a la cabecera Content-Type.

Entre 1991-1995, las cabeceras se introdujeron con un enfoque de prueba y error. Un servidor y un navegador añadían cada característica para ver su funcionalidad. Los problemas de interoperabilidad eran comunes. En un esfuerzo por resolver estos problemas, se publicó un documento informativo que describía las prácticas comunes en noviembre de 1996: RFC 1945 y se definió la versión HTTP/1.0 de manera estandarizada.

2.3. HTTP/1.1

Publicada a principios de 1997, esta versión clarificó ambigüedades e introdujo numerosas ventajas:

- Una conexión podía ser reutilizada, lo que ahorra tiempo. Ya no era necesario abrirla múltiples veces para mostrar los recursos incrustados en el único documento original.
- Se añadió la canalización (*pipelining*). Esto permitía enviar una segunda solicitud antes de que la respuesta a la primera se hubiera transmitido por completo. Esto reducía la latencia de la comunicación.
- También se admitieron las respuestas fragmentadas (*chunked responses*).
- Se introdujeron mecanismos adicionales de control de caché.
- Se introdujo la negociación de contenido, incluyendo idioma, codificación y tipo. Un cliente y un servidor podían ahora acordar qué contenido intercambiar.
- Gracias a la cabecera Host, la capacidad de alojar diferentes dominios desde la misma dirección IP permitía la colocación de servidores (server collocation).

Recordar que establecer una conexión TCP es una parte costosa del intercambio cliente-servidor y el inicio lento de TCP (las conexiones más longevas son más rápidas que las recién creadas) dio la necesidad de reutilizar la conexión para múltiples solicitudes y respuestas, evitando así tener que crear una nueva conexión para cada solicitud.

Sin embargo, los clientes aún tenían que esperar a que cada recurso se descargara antes de solicitar el siguiente (*Head-of-line blocking*). Para solucionar esto, la mayoría de los navegadores permiten hasta seis

conexiones TCP por sitio web (u origen). Con seis conexiones paralelas, los navegadores pueden obtener múltiples recursos simultáneamente usando el modelo HTTP/1.1, lo que ha añadido mejoras significativas de rendimiento.

Esta versión se publicó por primera vez como RFC 2068 en enero de 1997 y actualmente se tiene como RFC 2616 desde junio de 1999.

2.4. HTTP/2

Con los años, las páginas web se volvieron más complejas. Algunas eran incluso aplicaciones en sí mismas. Se mostraban más medios visuales y el volumen y tamaño de los scripts que añadían interactividad también aumentaron. Se transmitían muchos más datos a través de muchas más solicitudes HTTP y esto creaba más complejidad y sobrecarga para las conexiones HTTP/1.1.

Google implementó un protocolo experimental SPDY a principios de la década de 2010. Esta forma alternativa de intercambiar datos entre cliente y servidor acumuló interés de los desarrolladores que trabajaban tanto en navegadores como en servidores. SPDY definió un aumento en la capacidad de respuesta y resolvió el problema de la transmisión duplicada de datos, sirviendo como base para el protocolo HTTP/2.

El protocolo HTTP/2 difiere de HTTP/1.1 en varios aspectos:

- Es un protocolo binario en lugar de un protocolo de texto. No puede ser leído ni creado manualmente. A pesar de este obstáculo, permite la implementación de técnicas de optimización mejoradas.
- Es un protocolo multiplexado. Se pueden realizar solicitudes paralelas sobre la misma conexión, eliminando las restricciones del protocolo HTTP/1.x.
- Comprime cabeceras. Como estas son a menudo similares entre un conjunto de solicitudes, esto elimina la duplicación y la sobrecarga de los datos transmitidos.

Oficialmente estandarizado en mayo de 2015, el uso de HTTP/2 alcanzó su punto máximo en enero de 2022 con un 46.9 % de todos los sitios web y el RFC 9113 tiene su especificación formal. Los sitios web de alto tráfico mostraron la adopción más rápida en un esfuerzo por ahorrar en la sobrecarga de transferencia de datos y los presupuestos subsecuentes.

Esta rápida adopción se debió probablemente a que HTTP/2 no requería cambios en los sitios web ni en las aplicaciones. Para usarlo, solo se necesitaba un servidor actualizado que se comunicara con un navegador reciente. Solo se necesitó un conjunto limitado de grupos para desencadenar la adopción y a medida que las versiones heredadas de navegadores y servidores se renovaban, el uso aumentaba naturalmente, sin un trabajo significativo para los desarrolladores web.

2.5. HTTP/3 - HTTP sobre QUIC

Esta versión tiene la misma semántica que las versiones anteriores de HTTP pero utiliza **QUIC** en lugar de TCP para la porción de la capa de transporte. Para octubre de 2022, el 26 % de todos los sitios web utilizaban HTTP/3.

QUIC está diseñado para proporcionar una latencia mucho menor para las conexiones HTTP. Al igual que HTTP/2, es un protocolo multiplexado, pero HTTP/2 se ejecuta sobre una única conexión TCP, por lo que la detección de pérdida de paquetes y la retransmisión gestionadas en esta capa pueden bloquear todos los flujos.

QUIC ejecuta múltiples flujos sobre UDP e implementa la detección de pérdida de paquetes y la retransmisión de forma independiente para cada flujo, de modo que si ocurre un error, solo se bloquea el flujo con datos en ese paquete.

Definido en RFC 9114, HTTP/3 es compatible con la mayoría de los principales navegadores, incluidos Chromium (y sus variantes como Chrome y Edge) y Firefox.

3. Componentes de los sistemas basados en HTTP

HTTP es un protocolo cliente-servidor: las solicitudes son enviadas por una entidad, el *user-agent* (o un proxy en su nombre). La mayoría de las veces, el *user-agent* es un navegador web.

Cada solicitud individual se envía a un servidor, que la maneja y proporciona una contestación denominada respuesta. Entre el cliente y el servidor existen numerosas entidades, denominadas colectivamente *proxies*, que realizan diferentes operaciones y actúan como gateways o cachés.

3.1. User-agent

Es cualquier herramienta que actúa en nombre del usuario. Este rol es desempeñado principalmente por el navegador web, pero también puede ser desempeñado por programas para depurar aplicaciones. El navegador es siempre la entidad que inicia la solicitud. Nunca es el servidor (aunque se han añadido algunos mecanismos a lo largo de los años para simular mensajes iniciados por el servidor).

Para mostrar una página web, el navegador envía una solicitud original para obtener el documento HTML que representa la página. Luego, analiza este archivo, realizando solicitudes adicionales correspondientes a scripts de ejecución, información de diseño (CSS) para mostrar y sub-recursos contenidos dentro de la página (generalmente imágenes y videos).

El navegador web luego combina estos recursos para presentar el documento completo, la página web. Los scripts ejecutados por el navegador pueden obtener más recursos en fases posteriores y el navegador actualiza la página web en consecuencia.

Una página web es un documento de hipertexto. Esto significa que algunas partes del contenido mostrado son enlaces, que pueden ser activados (generalmente con un clic del ratón) para obtener una nueva página web, permitiendo al usuario dirigir su *user-agent* y navegar a través de la Web.

El navegador traduce estas directrices en solicitudes HTTP, e interpreta además las respuestas HTTP para presentar al usuario una respuesta clara.

3.2. Servidor Web

Un servidor aparenta ser virtualmente una única máquina; pero en realidad puede ser una colección de servidores que comparten la carga u otro software, generando total o parcialmente el documento bajo demanda.

Un servidor no es necesariamente una única máquina, sino que varias instancias de software de servidor pueden estar alojadas en la misma máquina. Con la versión HTTP/1.1 y la cabecera `Host`, pueden incluso compartir la misma dirección IP.

3.3. Proxies

Entre el navegador web y el servidor, numerosas computadoras y máquinas retransmiten los mensajes HTTP. Debido a la estructura en capas de la pila de la Web, la mayoría de estos operan en los niveles de transporte, red o físico, volviéndose transparentes en la capa HTTP y teniendo potencialmente un impacto significativo en el rendimiento.

Aquellos que operan en las capas de aplicación se denominan generalmente *proxies*. Estos pueden ser transparentes, reenviando las solicitudes que reciben sin alterarlas de ninguna manera, o no transparentes, en cuyo caso cambiarán la solicitud de alguna manera antes de pasarla al servidor.

Los proxies pueden realizar numerosas funciones:

- **Almacenamiento en caché (caching):** (la caché puede ser pública o privada, como la caché del navegador)
- **Filtrado (filtering):** (como un escaneo antivirus o controles parentales)
- **Balanceo de carga (load balancing):** (para permitir que múltiples servidores atiendan diferentes solicitudes)
- **Autenticación (authentication):** (para controlar el acceso a diferentes recursos)
- **Registro (logging):** (permitiendo el almacenamiento de información histórica)

4. Características de los servidores HTTP

HTTP está generalmente diseñado para ser legible por humanos, incluso con la complejidad añadida introducida en HTTP/2 al encapsular mensajes HTTP en tramas. Los mensajes HTTP pueden ser leídos y comprendidos por humanos, proporcionando pruebas más fáciles para los desarrolladores y una complejidad reducida para los recién llegados.

Introducidas en HTTP/1.0, las cabeceras HTTP hacen que este protocolo sea fácil de extender y experimentar. Incluso se puede introducir nueva funcionalidad mediante un acuerdo entre un cliente y un servidor sobre la semántica de una nueva cabecera.

HTTP es sin estado (stateless): no existe vínculo entre dos solicitudes que se llevan a cabo sucesivamente en la misma conexión. Esto tiene inmediatamente la perspectiva de ser problemático para los usuarios que intentan interactuar con ciertas páginas de manera coherente. Pero mientras que el núcleo de HTTP en sí mismo es sin estado, las cookies HTTP permiten el uso de sesiones con estado (stateful). Usando la extensibilidad de las cabeceras, las Cookies HTTP se añaden al flujo de trabajo, permitiendo que la creación de sesiones en cada solicitud HTTP comparta el mismo contexto, o el mismo estado.

Una conexión se controla en la capa de transporte y, por lo tanto, está fundamentalmente fuera del ámbito de HTTP. HTTP no requiere que el protocolo de transporte subyacente esté basado en conexión; solo requiere que sea fiable, o que no pierda mensajes (como mínimo, presentando un error en tales casos). Entre los dos protocolos de transporte más comunes en Internet, TCP es fiable y UDP no lo es. HTTP, por lo tanto, se basa en el estándar TCP, que está basado en conexión.

Antes de que un cliente y un servidor puedan intercambiar un par de solicitud/respuesta HTTP, deben establecer una conexión TCP, un proceso que requiere varios viajes de ida y vuelta (round-trips). El comportamiento predeterminado de HTTP/1.0 es abrir una conexión TCP separada para cada par de solicitud/respuesta HTTP. Esto es menos eficiente que compartir una única conexión TCP cuando se envían múltiples solicitudes en rápida sucesión.

Para mitigar este defecto, HTTP/1.1 introdujo la canalización (pipelining, que resultó difícil de implementar) y las conexiones persistentes: la conexión TCP subyacente puede controlarse parcialmente usando la cabecera `Connection`.

HTTP/2 fue un paso más allá al multiplexar (multiplexing) mensajes sobre una única conexión, ayudando a mantener la conexión *caliente* y más eficiente.

Esta naturaleza extensible de HTTP ha permitido, con el tiempo, un mayor control y funcionalidad de la Web. Los métodos de caché y autenticación fueron funciones manejadas temprano en la historia de HTTP. La capacidad de relajar la restricción de origen, por el contrario, solo se añadió en la década de 2010, cuando las aplicaciones web comenzaron a necesitarlo. Otras características que se han añadido incluyen:

- **Almacenamiento en caché (Caching):** Cómo se almacenan en caché los documentos puede ser controlado por HTTP. El servidor puede instruir a los proxies y clientes sobre qué almacenar en caché y por cuánto tiempo. El cliente puede instruir a los proxies de caché intermedios que ignoren el documento almacenado.
- **Relajación de la restricción de origen:** Para prevenir el espionaje (snooping) y otras invasiones de la privacidad, los navegadores web imponen una separación estricta entre sitios web. Solo las páginas del mismo origen (same origin) pueden acceder a toda la información de una página web. Aunque tal restricción es una carga para el servidor, las cabeceras HTTP pueden relajar esta separación estricta en el lado del servidor, permitiendo que un documento se convierta en un mosaico (patchwork) de información proveniente de diferentes dominios; incluso podría haber razones de seguridad para hacerlo.
- **Autenticación:** Algunas páginas pueden estar protegidas para que solo usuarios específicos puedan acceder a ellas. La autenticación básica puede ser proporcionada por HTTP, ya sea usando las

cabeceras `WWW-Authenticate` y similares, o estableciendo una sesión específica usando cookies HTTP.

- **Proxy y tunelización (tunneling):** Los servidores o clientes a menudo se encuentran en intranets y ocultan su verdadera dirección IP a otras computadoras. Las solicitudes HTTP luego pasan a través de proxies para cruzar esta barrera de red. No todos los proxies son proxies HTTP. El protocolo SOCKS, por ejemplo, opera a un nivel inferior. Otros protocolos, como ftp, pueden ser manejados por estos proxies.
- **Sesiones:** Usar cookies HTTP permite vincular solicitudes con el estado del servidor. Esto crea sesiones, a pesar de que HTTP básico es un protocolo sin estado (state-less). Esto es útil no solo para las cestas de la compra de comercio electrónico, sino también para cualquier sitio que permita la configuración de la salida por parte del usuario.

5. Flujo HTTP

Cuando un cliente desea comunicarse con un servidor, ya sea el servidor final o un proxy intermedio, realiza los siguientes pasos:

1. **Abrir una conexión TCP:** La conexión TCP se utiliza para enviar una solicitud, o varias, y recibir una respuesta. El cliente puede abrir una nueva conexión, reutilizar una conexión existente, o abrir varias conexiones TCP a los servidores.
2. **Enviar un mensaje HTTP:** Los mensajes HTTP (antes de HTTP/2) son legibles por humanos. Con HTTP/2, estos mensajes se encapsulan en tramas, haciéndolos imposibles de leer directamente, pero el principio sigue siendo el mismo.
3. **Cerrar o reutilizar la conexión** para futuras solicitudes.

Si la canalización (pipelining) de HTTP está activada, se pueden enviar varias solicitudes sin esperar a que la primera respuesta se reciba por completo. La canalización de HTTP ha demostrado ser difícil de implementar en las redes existentes, donde coexisten piezas antiguas de software con versiones modernas. La canalización de HTTP ha sido reemplazada en HTTP/2 con una multiplexación (multiplexing) más robusta de solicitudes dentro de una trama.

6. Mensajes HTTP

Los mensajes HTTP, tal como se definen en HTTP/1.1 y anteriores, son legibles por humanos. En HTTP/2, estos mensajes están embebidos en una estructura binaria, una trama, permitiendo optimizaciones como la compresión de cabeceras y la multiplexación. Incluso si solo se envía parte del mensaje HTTP original en esta versión de HTTP, la semántica de cada mensaje no cambia y el cliente reconstituye (virtualmente) la solicitud HTTP/1.1 original. Por lo tanto, es útil comprender los mensajes HTTP/2 en el formato HTTP/1.1.

Existen dos tipos de mensajes HTTP, solicitudes (requests) y respuestas (responses), cada uno con su propio formato.

6.1. Solicitudes

Las solicitudes constan de los siguientes elementos:

- Un **método HTTP**, usualmente un verbo como GET, POST, o un sustantivo como OPTIONS o HEAD que define la operación que el cliente desea realizar. Típicamente, un cliente desea obtener un recurso (usando GET) o publicar (post) el valor de un formulario HTML (usando POST), aunque se pueden necesitar más operaciones en otros casos.
- La **ruta (path) del recurso** a obtener; la URL del recurso despojada de elementos que son obvios por el contexto, por ejemplo, sin el protocolo (`http://`), el dominio (aquí, `developer.mozilla.org`), o el puerto TCP (aquí, 80).
- La **versión del protocolo HTTP**.
- **Cabeceras opcionales** que transmiten información adicional para los servidores.
- Un **cuerpo (body)**, para algunos métodos como POST, similar a los de las respuestas, que contienen el recurso enviado.

6.2. Respuestas

Las respuestas constan de los siguientes elementos:

- La **versión del protocolo HTTP** que siguen.
- Un **código de estado (status code)**, que indica si la solicitud fue exitosa o no, y por qué.
- Un **mensaje de estado (status message)**, una breve descripción no autoritativa (non-authoritative) del código de estado.
- **Cabeceras HTTP**, como las de las solicitudes.
- Opcionalmente, un **cuerpo (body)** que contiene el recurso obtenido.

7. APIs basadas en HTTP

La API más comúnmente utilizada basada en HTTP es la **Fetch API**, que puede usarse para realizar solicitudes HTTP desde JavaScript. La Fetch API reemplaza a la API `XMLHttpRequest`.

Otra API, **server-sent events** (eventos enviados por el servidor), es un servicio unidireccional que permite a un servidor enviar (push) eventos al cliente, utilizando HTTP como mecanismo de transporte. Usando la interfaz `EventSource`, el cliente abre una conexión y establece manejadores de eventos (event handlers). El navegador del cliente convierte automáticamente los mensajes que llegan en el flujo (stream) HTTP en los objetos `Event` apropiados. Luego los entrega a los manejadores de eventos que se han registrado para el type de los eventos si se conoce, o al manejador de eventos `onmessage` si no se estableció ningún manejador de eventos específico del tipo.

Los servidores web pueden alojar múltiples sitios web y manejar muchas solicitudes simultáneamente. Algunos de los servidores web más populares incluyen Apache HTTP Server (Tomcat), Nginx, Microsoft Internet Information Services (IIS) y Sun Java System Web Server.

Referencias

- [1] B. A. Forouzan, *Data Communications and Networking*. McGraw-Hill, 2003.
- [2] D. E. Comer, *Internetworking with TCP/IP Vol. 1: Principles, Protocols, and Architecture*. Prentice Hall, 2000.
- [3] C. Hunt, *TCP/IP Network Administration*. O'Reilly Media, 2002.
- [4] J. Edwards y R. Bramante, *Networking Self-Teaching Guide OSI, TCP/IP, LANs, MANs, WANs, Implementation, Management, and Maintenance*. Wiley, 2009.
- [5] M. Burgess, *Principles of Network and System Administration*. Wiley, 2004.
- [6] K. R. Fall y W. R. Stevens, *TCP/IP Illustrated, Volume 1: The Protocols*. Pearson, 2012.
- [7] K. Siyan y T. Parker, *TCP/IP Unleashed*. Sams Publishing, 2002.
- [8] C. M. Kozierok, *The TCP/IP Guide*. Charles M. Kozierok, 2005.